Multi-model

As stated earlier, the OrientDB engine has support for various models, as listed below:

- Graph
- Document
- Key-value
- Object model

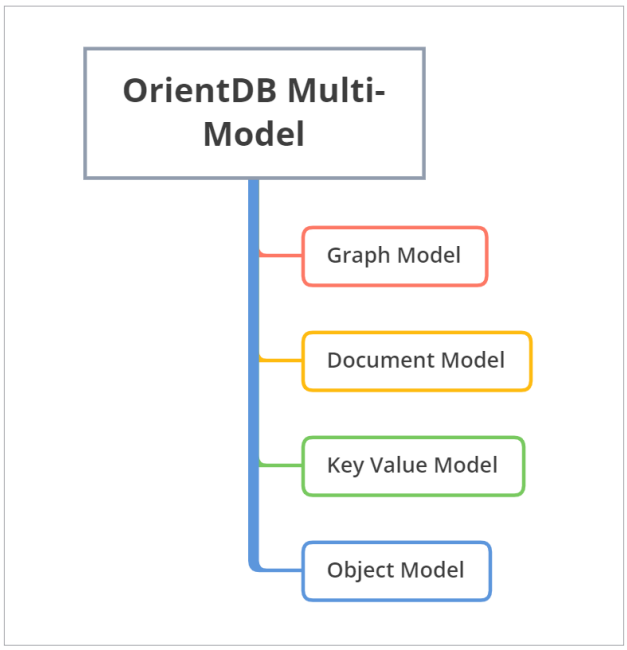

Figure 5: Multi-model OrientDB

The real power of OrientDB lies in the fact that it can combine all the features of the aforementioned four models. The core engine itself is built to support all these models. Some multi-model choices simply provide an interface. At the core level these tools have only one model, but through some sort of an additional API layer, they provide support for other models. This hinders their performance by affecting the speed and scalability. In the case of OrientDB, the multi-model support is at the core level, which enhances its speed and scalability.

The document model: Here, data is stored inside the documents. These documents are not mandated to have any specific schema. They are flexible and easy to modify. OrientDB uses *Class*, *Document* and *Link* to represent *Table*, *Row* and *Relationship* respectively.

The graph model: In the graph model, vertices and edges are used. The vertices are connected by edges. Edges are used to link two vertices. The OrientDB graph model has a *Class* that extends *Vertices* and *Edges*.

Key-value model: The key-value model is the simplest of the models. The OrientDB key-value model has *Class* for *Table* and *Link* for *Relationships*.

Object model: The object model has support for inheritance between types. With features such as direct binding, this model is highly suitable in scenarios where object representation can capture the real-world scenario in a better manner.

PyOrient

One of the advantages of OrientDB is the support for multiple languages. With the PyOrient module, it can be used along with the Python language. To install PyOrient, use the following code snippet:

```
sudo pip install pyorient
```

The PyOrient client can be used to connect to the OrientDB server, as follows:

```
client = pyorient.OrientDB("localhost", 2424)
session_id = client.connect("admin", "admin_passwd")
```

The *shutdown* operation can be carried out with the following code snippet:

```
client.shutdown('root', 'root_passwd')
```

A few useful functions to work with PyOrient are listed below.

- *command():* To issue SQL commands
- *Batch():* To issue batches of commands
- *db_create():* For creating a database
- *db_drop():* For removing a database
- *query ():* To issue synchronous queries to the database.

For the detailed list, navigate to *https://orientdb.com/docs/2.2.x/PyOrient-Client.html.*

With all these features, OrientDB is a great choice if you want multi-model support for your application. If you are interested in learning more about OrientDB, there is a free course offered. You can find out more about the course at *https://orientdb.org/getting-started.* END

References

[1] OrientDB official site: *https://orientdb.org/*
[2] Documentation: *http://www.orientdb.com/docs/last/index.html*

By: Dr K.S. Kuppusamy

The author is an assistant professor of computer science, School of Engineering and Technology, Pondicherry Central University, Pondicherry. He has more than 15 years of teaching and research experience in academia and industry. His research interests include accessible computing, Web information retrieval, and mobile computing.